DOCAPOSTE Institute

FORMATION · 2 JOURS · CLASSE VIRTUELLE

GitHub Copilot Débutant

Guide des 8 ateliers individuels · fil rouge TaskFlow

Houssem Eddine LASSOUED · Docaposte Institute

Version du 5 novembre 2026 · dépôt `taskflow-copilot`

Mode d’emploi

Pendant deux jours, vous construisez **TaskFlow**, un petit outil de gestion de tickets (backlog et Kanban), avec GitHub Copilot dans VS Code. Chaque atelier ajoute une brique au même dépôt. Les ateliers sont **individuels** : chacun sur son poste, minuteur affiché, points d’étape oraux toutes les 15 à 20 minutes. Bloqué plus de 5 minutes ? Dites-le au micro : partage d’écran de 2 minutes.

Installer

1. **Node.js 22 ou plus** : `node --version`. Aucune dépendance à installer : `npm install` n'est pas nécessaire.
2. **VS Code** à jour, connecté à **GitHub Copilot** (icône Copilot dans la barre d'état).
3. Récupérez le dépôt, au choix :
   - `git clone https://github.com/houssemeddinelassoued/taskflow-copilot.git`
   - ou **Code › Download ZIP** sur https://github.com/houssemeddinelassoued/taskflow-copilot, puis décompressez.

   Dans VS Code : Fichier › Ouvrir le dossier › `taskflow-copilot`. Le guide PDF des 8 ateliers est à la racine : `Guide-des-ateliers-Copilot-Debutant.pdf`.
4. `npm test` : des tests échouent, **c'est normal** : vous les ferez passer au vert pendant les ateliers.
5. Facultatif, pour les variantes : Python 3.10+ et/ou un JDK 21+.

Commandes

Commande	Effet
`npm test`	tous les tests du dossier `test/`
`npm run test:atelier4` (5, 6)	les tests d'acceptation d'un atelier
`npm run test:revue`	les tests de la revue (Atelier 8)
`npm run couverture`	tests + couverture de code
`npm run demo`	aperçu du backlog dans le terminal
`npm run serveur`	l'interface Kanban sur http://localhost:3000

Vue d’ensemble

#	Atelier	Module	Durée
1	Premier contact avec Copilot	1 · Introduction	20 min
2	Configurer son poste et prendre les réflexes clavier	2 · Installation et configuration	30 min
3	Le contexte fait la suggestion	3 · Comprendre les suggestions	35 min
4	TaskFlow v1 : le modèle de ticket	4 · Génération de code simple	55 min
5	TaskFlow v2 : backlog, classe et import	5 · Fonctions et blocs de code	50 min
6	Refactorer le Kanban legacy	6 · Refactoring	45 min
7	Une vraie suite de tests, générée et vérifiée	7 · Tests unitaires	50 min
8	Revue de code et livraison	8 · Bonnes pratiques et vérification	40 min

Règles du jeu

- **Données fictives uniquement** : aucun code client, aucun secret, aucune donnée personnelle dans un prompt.
- **Lire avant d'accepter** : une suggestion acceptée devient *votre* code.
- **Un test au vert vaut mieux qu'une explication convaincante.**
- Bloqué plus de 5 minutes ? Dites-le au micro au prochain point d'étape : partage d'écran de 2 minutes.

ATELIER 1

MODULE 1 · INTRODUCTION À GITHUB COPILOT

Premier contact avec Copilot

⏱ 20 min | Jour 1 · 10 h 30 | Individuel

`katas/01-premiers-pas.js`, `JOURNAL.md`

OBJECTIF

Voir Copilot travailler sur un vrai dépôt : lui faire **expliquer** le projet TaskFlow, puis obtenir vos premières **suggestions inline**, et noter ce qui est juste… et ce qui ne l'est pas.

Point de départ

Le dépôt `taskflow-copilot` est ouvert dans VS Code (Fichier › Ouvrir le dossier), Copilot est connecté (icône Copilot dans la barre d'état, sans point d'exclamation).

Étapes

1. Ouvrez la vue Chat (`Ctrl+Alt+I`), choisissez l'agent **Ask** et demandez : « Explique la structure de ce projet en 5 lignes : à quoi sert chaque dossier ? Cite les fichiers. »
2. Comparez sa réponse avec `README.md`. Notez dans `JOURNAL.md` **une** affirmation exacte et, si vous en trouvez une, **une** affirmation inventée ou imprécise.
3. Ouvrez `katas/01-premiers-pas.js`. Pour chaque ligne marquée ▶ : attendez le texte grisé, **lisez-le**, puis `Tab` (accepter) ou `Échap` (refuser).
4. Bloc 4 (année bissextile) : vérifiez la suggestion sur 1900, 2000 et 2024. Demandez dans le chat : « Teste mentalement estBissextile(1900). »
5. Complétez la ligne « Atelier 1 » de `JOURNAL.md` : ce qui vous a surpris, ce que vous avez refusé, et pourquoi.

C'EST TERMINÉ QUAND…

- [ ] `JOURNAL.md` contient vos 3 observations de l'atelier 1.
- [ ] Les 4 blocs de `katas/01-premiers-pas.js` sont complétés **et relus**.
- [ ] Vous savez dire si `estBissextile(1900)` renvoie la bonne valeur.

PIÈGES À OBSERVER

- Copilot répond avec aplomb, même quand il devine : une explication fluide n'est pas une preuve.
- Le bloc 3 (tableau de jours) montre la complétion de motifs : très efficace sur ce qui est répétitif.
- La règle des années bissextiles est un classique où une suggestion « presque juste » passe inaperçue (divisible par 4, **sauf** par 100, **sauf** par 400).

BONUS

Posez la même question qu'à l'étape 1 dans une **nouvelle** conversation (`Ctrl+N` dans le chat). La réponse est-elle identique ? Notez-le : Copilot n'est pas déterministe.

COINCÉ ?

Aucun texte grisé n'apparaît ? Vérifiez l'icône Copilot de la barre d'état (clic → « Inline suggestions » activé pour JavaScript), puis forcez une suggestion avec `Alt+\` . Toujours rien : point d'étape au micro, partage d'écran 2 minutes.

ATELIER 2

MODULE 2 · INSTALLATION ET CONFIGURATION

Configurer son poste et prendre les réflexes clavier

⏱ 30 min | Jour 1 · 11 h 45 | Individuel

`.vscode/settings.json` , `katas/02-raccourcis.js`

OBJECTIF

Vérifier votre licence, régler Copilot pour **votre** façon de travailler (langue, langages actifs, suggestions de modification suivante) dans un fichier `.vscode/settings.json` versionné, puis enchaîner les **8 gestes clavier** essentiels sans toucher la souris.

Point de départ

Atelier 1 terminé. Dépôt `taskflow-copilot` ouvert dans VS Code.

Étapes

1. **Licence** : cliquez sur l'icône Copilot de la barre d'état. Notez votre plan (Free, Pro, Business…) et, si vous êtes en Free, le quota de complétions restant du mois.
2. **Réglages** : `Ctrl+Maj+P` › « Préférences : Ouvrir les paramètres de l'espace de travail (JSON) ». Ajoutez :

```
{
  "github.copilot.chat.localeOverride": "fr",
  "github.copilot.enable": { "*": true, "markdown": false,
"plaintext": false },
  "github.copilot.nextEditSuggestions.enabled": true,
  "editor.inlineSuggest.enabled": true
}
```

3. Vérifiez chaque réglage par son **effet visible** : le chat répond en français ; aucun texte grisé dans `README.md` ; une flèche apparaît dans la gouttière après un renommage.
4. **Parcours clavier** : ouvrez `katas/02-raccourcis.js` et réalisez les gestes 1 à 8 dans l'ordre. Cochez ci-dessous chaque geste réussi **sans souris**.

Geste	Raccourci	Fait
1 · accepter	`Tab`	☐
2 · refuser	`Échap`	☐
3 · accepter mot par mot	`Ctrl+→`	☐
4 · alternatives	`Alt+]` / `Alt+[`	☐
5 · déclencher	`Alt+\`	☐
6 · chat en ligne	`Ctrl+I`	☐
7 · vue Chat	`Ctrl+Alt+I`	☐
8 · modification suivante	`Tab` (flèche de gouttière)	☐

C'EST TERMINÉ QUAND…

- ☐ `.vscode/settings.json` contient vos 4 réglages et chacun a été vérifié par son effet.
- ☐ Les 8 gestes sont cochés.
- ☐ `JOURNAL.md` : votre plan Copilot et le geste qui vous sera le plus utile.

PIÈGES À OBSERVER

- `github.copilot.enable` désactive l'inline pour un langage, **pas** le chat.
- macOS : `Ctrl` devient `Cmd` et `Alt` devient `Option` (sauf `Ctrl+Alt+I` → `Ctrl+Cmd+I`).
- Sur un clavier AZERTY, `Alt+]` et `Alt+\` peuvent ne pas répondre : survolez le texte grisé (barre « 1/3 ») ou réattribuez le raccourci.

BONUS

Ajoutez `"editor.inlineSuggest.edits.showCollapsed": true` : les suggestions de modification suivante restent repliées tant que vous n'y allez pas. Puis `Ctrl+K Ctrl+S` , « inline suggest » : attribuez un raccourci à « accepter la ligne suivante ».

COINCÉ ?

Les réglages n'ont pas d'effet ? Vérifiez que le fichier est bien `.vscode/settings.json` **à la racine du dépôt** et que le JSON est valide (pas de virgule finale). Le chat reste en anglais : rechargez la fenêtre (`Ctrl+Maj+P` › « Recharger la fenêtre »).

ATELIER 3

MODULE 3 · COMPRENDRE LES SUGGESTIONS

Le contexte fait la suggestion

⏱ 35 min · Jour 1 · 14 h 15 · Individuel

`katas/03-contexte/` , `JOURNAL.md`

OBJECTIF

Prouver par l'expérience que la qualité d'une suggestion dépend de ce que Copilot **voit** : nom seul, fichier voisin ouvert, commentaire précis, import. Puis repérer, **en lisant**, trois défauts dans du code accepté trop vite.

Point de départ

Ateliers 1 et 2 terminés. Dans `JOURNAL.md`, la grille « Atelier 3 » est prête à remplir.

Étapes

1. **Exp. 1 · nom seul** : fermez tous les onglets, ouvrez `experience-1-nom-seul.js`, placez le curseur dans `calculerVelocite`. Notez la suggestion (propriétés inventées ? gestion du vide ?).
2. **Exp. 2 · avec contexte** : ouvrez `sprints-exemple.js` dans un onglet voisin, puis `experience-2-avec-contexte.js`. Même fonction, JSDoc précise : comparez. Exécutez `node katas/03-contexte/experience-2-avec-contexte.js` : le résultat attendu est `21.7`.
3. **Exp. 3 · commentaires** : dans `experience-3-commentaires.js`, comparez le bloc A (vague) et le bloc B (précis). Puis décommentez l'import du bloc C et recommencez le bloc A : que change l'import ?
4. **Alternatives** : sur le bloc B, parcourez les alternatives (`Alt+]` ou survol). Gardez celle qui respecte la contrainte d'heure locale.
5. **Exp. 4 · lire avant d'accepter** : trouvez les 3 défauts de `experience-4-lire-avant-accepter.js` **avant** de l'exécuter. Écrivez votre diagnostic, puis exécutez le fichier et demandez à Copilot Chat (Ask) son avis. Corrigez.

C'EST TERMINÉ QUAND…

- [ ] La grille « Atelier 3 » de `JOURNAL.md` est remplie pour les 4 expériences (suggestion obtenue, ce qui a changé, pourquoi).
- [ ] `experience-2-avec-contexte.js` affiche `21.7` .
- [ ] `experience-4-lire-avant-accepter.js` affiche `3` , `false` , `5` .

PIÈGES À OBSERVER

- L'expérience 2 ne marche bien que si l'onglet voisin est **ouvert** : un fichier fermé n'est pas dans le contexte de la suggestion inline.
- Le bloc C (import de `date-fns`) pousse Copilot vers une bibliothèque **qui n'est pas installée** : une suggestion peut être parfaite… et inutilisable.
- `estPrioritaire` affecte au lieu de comparer : la suggestion « a l'air » juste.

BONUS

Refaites l'expérience 1 en changeant seulement le **nom du fichier** en `velocite-sprints.js` . Le nom du fichier fait-il partie du contexte ? Notez votre conclusion.

COINCÉ ?

Les suggestions des expériences 1 et 2 sont identiques ? Vérifiez que l'onglet `sprints-exemple.js` est ouvert **dans le même groupe d'éditeurs**, puis retapez la première ligne du corps de la fonction pour relancer une suggestion.

ATELIER 4

MODULE 4 · GÉNÉRATION DE CODE SIMPLE

TaskFlow v1 : le modèle de ticket

⏱ 55 min | Jour 1 · 15 h 55 | Individuel

`src/tickets.js` , `test/tickets.test.js` , `katas/python/` , `katas/java/`

OBJECTIF

Générer avec Copilot, **commentaire par commentaire**, le cœur de données de TaskFlow : constantes du domaine, `estPointsValide` , `validerTicket` et `creerTicket` . Les 8 tests d'acceptation fournis doivent passer au vert, sans jamais modifier les tests.

Point de départ

`src/tickets.js` ne contient que des commentaires-guides. `npm run test:atelier4` échoue : c'est normal.

Étapes

1. Lancez `npm run test:atelier4` et lisez les échecs : ce sont vos spécifications exécutables.
2. **Constantes** : curseur sous le commentaire 1, laissez Copilot proposer. Vérifiez `Object.freeze` et l'ordre des statuts. Acceptez mot par mot si besoin (`Ctrl+→`).
3. **estPointsValide** : écrivez seulement `export function estPointsValide(points) {` et laissez compléter. Vérifiez à la main les cas `"5"` et `undefined` .
4. **validerTicket** puis **creerTicket** : une fonction à la fois, test après chaque fonction. Si une suggestion s'arrête à la première erreur, refusez-la et précisez le commentaire.
5. Quand tout est vert : `npm run demo` affiche les tickets ; faites un commit « Atelier 4 : modèle de ticket ».
6. **Variante au choix** (si vous avez fini) : même cahier en Python (`katas/python/tickets.py` , tests : `python -m unittest katas/python/test_tickets.py`) ou en Java (`java katas/java/Ticket.java` → « 6/6 OK »).

C'EST TERMINÉ QUAND…

- [ ] `npm run test:atelier4` : **8 tests au vert**, tests non modifiés.
- [ ] `src/tickets.js` ne contient aucun `new Date()` .
- [ ] Un commit « Atelier 4 : modèle de ticket ».

PIÈGES À OBSERVER

- Copilot propose volontiers `points > 0 && points <= 13` : 4 passerait. La suite 1-2-3-5-8-13 est une **liste**, pas un intervalle.
- `typeof titre === 'string'` doit être vérifié **avant** `trim()` , sinon `titre: undefined` fait planter au lieu de renvoyer « titre invalide ».
- En Python, `True == 1` : `est_points_valide(True)` doit être faux. Copilot l'oublie presque toujours.

BONUS

Demandez dans le chat en ligne (`Ctrl+I` , sélection sur `validerTicket`) : « Ajoute une règle : un bug doit avoir la priorité 1 ou 2. » Puis écrivez **vous-même** le test qui le prouve.

COINCÉ ?

Après 45 minutes, si `validerTicket` résiste : copiez `reprises/fin-atelier-04/tickets.js` dans `src/` pour ne pas bloquer le Jour 2, et notez dans `JOURNAL.md` ce qui a coincé.

ATELIER 5

MODULE 5 · GÉNÉRATION DE FONCTIONS ET BLOCS DE CODE

TaskFlow v2 : backlog, classe et import

⏱ 50 min | Jour 2 · 9 h 55 | Individuel

`src/backlog.js`, `src/api.js`, `test/backlog.test.js`, `test/api-csv.test.js`

OBJECTIF

Générer des **fonctions complètes** (paramètres, valeurs de retour, cas vides), une **classe** `Backlog` et un **parseur CSV**, chacun documenté en **JSDoc**. Les tests d'acceptation passent et l'interface web de TaskFlow s'allume.

Point de départ

Atelier 4 terminé (`npm run test:atelier4` au vert). Sinon : copiez `reprises/fin-atelier-04/tickets.js` dans `src/` .

Étapes

1. **Plan d'abord** : dans la vue Chat, agent **Ask**, joignez `#file:src/backlog.js` et demandez : « Propose les signatures JSDoc des 4 éléments décrits, sans écrire les corps. » Relisez le plan.
2. **trierBacklog, filtrerBacklog, statistiques** : pour chacune, écrivez `/**` + Entrée au-dessus de la signature (Copilot propose la JSDoc), puis laissez compléter le corps. Test après **chaque** fonction : `npm run test:atelier5` .
3. **Classe Backlog** : écrivez `export class Backlog {` et laissez Copilot proposer constructeur et méthodes, une méthode à la fois. Vérifiez que le constructeur **copie** le tableau.
4. **parserCsv** : générez-la depuis le commentaire. Ouvrez `data/import.csv` dans un onglet voisin pendant la génération. Le test attend **6 tickets et 2 rejets**.
5. **chargerTickets** : générez-la, puis lancez `npm run serveur` et ouvrez `http://localhost:3000` . Testez `chargerTickets('http://localhost:3000/api/panne')` dans le chat en ligne ou un petit script : l'erreur doit être « HTTP 500 ».
6. Commit « Atelier 5 : backlog, classe, import ».

C'EST TERMINÉ QUAND…

- [ ] `npm run test:atelier5` au vert (backlog **et** CSV).
- [ ] Chaque fonction exportée a sa JSDoc (`@param` , `@returns`).
- [ ] L'interface `http://localhost:3000` affiche le backlog trié, les filtres et les statistiques.

PIÈGES À OBSERVER

- `sort()` trie **sur place** : `tickets.sort(...)` modifie le tableau reçu. Exigez `[...tickets].sort(...)` ou `toSorted()` .
- La recherche sans accents demande `normalize('NFD')` : Copilot propose souvent `toLowerCase()` seul, et « securite » ne trouve plus « Sécurité ».
- `split(',')` au lieu de `split(';')` , ou un numéro de ligne qui oublie l'en-tête : le test des rejets le verra.
- L'avancement d'une liste vide : `0 / 0` donne `NaN` .

BONUS

Ajoutez `exporterCsv(tickets)` qui produit le format inverse, et vérifiez l'aller-retour `parserCsv(exporterCsv(liste))` dans un test. Que se passe-t-il avec un titre contenant « ; » ?

COINCÉ ?

Une fonction résiste plus de 10 minutes : passez à la suivante, les tests sont indépendants. Retour en fin d'atelier ; point de reprise : `reprises/fin-atelier-05/` .

ATELIER 6

MODULE 6 · REFACTORING AVEC COPILOT

Refactorer le Kanban legacy

⏱ 45 min | Jour 2 · 11 h 35 | Individuel

`src/legacy/kanban-legacy.js`, `test/kanban-legacy.test.js`, `src/kanban.js`, `test/kanban-contrat.test.js`

OBJECTIF

Transformer `traiter(l, i, s, opt)`, un code de 2019 illisible, en code clair **sans changer son comportement** (tests de caractérisation toujours verts), puis écrire sa version moderne `deplacerTicket` : pure, nommée, et **corrigée** du bug de limite WIP.

Point de départ

`npm run test:atelier6` : les 9 tests de caractérisation passent, les 3 tests de contrat échouent (`src/kanban.js` est vide).

Étapes

1. **Comprendre** : sélectionnez `traiter`, chat **Ask** : « Explique ce que fait cette fonction, règle par règle, et liste ses défauts. » Comparez avec les noms des tests de caractérisation.
2. **Renommer** : `F2` sur `l`, `i`, `s`, `r` → `tickets`, `id`, `versStatut`, `ticket`. Acceptez les suggestions de modification suivante (`Tab`). Tests.
3. **Simplifier et extraire** : chat en ligne (`Ctrl+I`) sur la fonction : « Remplace var par const/let, extrais trouverTicket et compterEnCours, garde exactement le même comportement, y compris == et la mutation. » Relisez le diff **avant** d'accepter. Tests.
4. **Comparer** : `git diff` (ou la vue Contrôle de code source) : chaque changement doit être un renommage, une extraction ou une simplification. Commit « refactor : kanban legacy lisible ».
5. **Réécrire** : générez `src/kanban.js` depuis ses commentaires-guides (`deplacerTicket`, pure, `===`, messages explicites). Le test de contrat « refuse le 4e ticket en cours » doit passer : c'est la **correction** du bug.
6. Commit séparé « fix : limite WIP respectée dans deplacerTicket ».

C'EST TERMINÉ QUAND…

- [ ] `npm run test:atelier6` : **12 tests au vert** (9 de caractérisation + 3 de contrat).
- [ ] `kanban-legacy.js` lisible, sans `var`, avec des fonctions extraites.
- [ ] Deux commits distincts : refactoring, puis correction.

PIÈGES À OBSERVER

- « Améliore ce code » produit souvent un refactoring **et** une correction mêlés : le test « ⚠ accepte un 4e ticket » passe alors au rouge. Refactoring = même comportement.
- Remplacer `==` par `===` dans le legacy casse le test « "1" trouve le ticket 1 » : c'est un changement de comportement, pas un nettoyage.
- Copilot peut supprimer la mutation `r.statut = s` en « améliorant » : relisez le diff ligne par ligne.

BONUS

Faites déléguer `traiter` à `deplacerTicket` (adaptateur), puis mettez à jour **délibérément** le test « ⚠ » : c'est la décision de corriger le comportement historique, à commiter seule avec un message clair.

COINCÉ ?

Un test de caractérisation passe au rouge et vous ne voyez pas pourquoi : `git diff`, puis annulez le dernier pas (`git checkout -- src/legacy/kanban-legacy.js`) et refaites-le en plus petit. Point de reprise : `reprises/fin-atelier-06/`.

ATELIER
7

MODULE 7 · TESTS UNITAIRES AVEC COPILOT

Une vraie suite de tests, générée et vérifiée

⏱ 50 min | Jour 2 · 14 h 30 | Individuel

`test/kanban.test.js`, `test/api-charger.test.js` (à créer)

OBJECTIF

Faire générer par Copilot une suite de tests **lisible** (Arrange-Act-Assert, noms en français qui décrivent le comportement) pour `deplacerTicket`, puis tester `chargerTickets` **sans réseau** grâce à un mock de `fetch`. Prouver ensuite que les tests détectent une erreur (test de mutation).

Point de départ

Atelier 6 terminé (`src/kanban.js` au vert sur le contrat). Sinon : `reprises/fin-atelier-06/kanban.js`.

Étapes

1. **Liste des cas d'abord** : chat **Ask**, `#file:src/kanban.js` : « Liste les cas à tester pour deplacerTicket : nominaux, limites, erreurs. Pas de code. » Complétez la liste vous-même (au moins 10 cas, dont : 3 en cours → 4e refusé, 2 en cours → 3e accepté, ticket non estimé, termineLe, immuabilité).
2. **Générer** : dans `test/kanban.test.js`, collez la liste en commentaire, puis `/tests` (chat en ligne) ou demandez la génération dans le chat. Imposez : `node:test`, `assert/strict`, un `describe` par règle, noms « refuse… / accepte… ».
3. **Vérifier chaque assertion** : pour chaque test, demandez-vous « échouerait-il si le code était faux ? ». Supprimez les tests tautologiques (qui recalculent le résultat avec la même logique que le code).
4. **Mock de fetch** : dans `test/api-charger.test.js`, testez `chargerTickets` avec un faux `fetchFn` : succès, `HTTP 500`, JSON qui n'est pas un tableau. Bonus : `mock.fn()` de `node:test` pour vérifier l'URL appelée.
5. **Mutation** : dans `src/kanban.js`, changez `>=` en `>` dans la règle WIP. Au moins un test doit passer au **rouge**. Sinon, écrivez le test manquant. Rétablissez.
6. `npm run couverture` : notez le pourcentage de lignes de `kanban.js` et `api.js`. Commit « test : suite kanban et api ».

C'EST TERMINÉ QUAND…

- [ ] `test/kanban.test.js` : au moins 10 tests, tous au vert, groupés par règle R1–R4.
- [ ] `test/api-charger.test.js` : 3 tests avec mock, aucun appel réseau.
- [ ] La mutation `>=` → `>` est détectée.

PIÈGES À OBSERVER

- Un test généré peut **réutiliser la logique du code** pour calculer l'attendu : il passe toujours, même si le code est faux.
- `assert.equal` sur deux objets compare les **références** : utilisez `assert.deepEqual` (avec `assert/strict`).
- Un mock trop bavard (qui vérifie chaque appel interne) casse au moindre refactoring : vérifiez le **résultat**, pas la recette.
- Une date « du jour » dans un test le rend instable : passez `aujourdHui` en paramètre.

BONUS

Demandez à Copilot un test « paramétré » : un tableau de cas `[statutDépart, versStatut, attendu]` parcouru par une boucle `for…of` avec un `it` par cas.

COINCÉ ?

`node --test` ne trouve pas votre fichier : il doit se terminer par `.test.js` et se trouver dans `test/`. Une erreur `ERR_MODULE_NOT_FOUND` : vérifiez le chemin `../src/kanban.js` (avec l'extension `.js`).

ATELIER 8

MODULE 8 · BONNES PRATIQUES ET VÉRIFICATION

Revue de code et livraison

⏱ 40 min | Jour 2 · 16 h 10 | Individuel

`ateliers/atelier-08-revue/` , `.github/copilot-instructions.md`

OBJECTIF

Relire huit suggestions Copilot « acceptées trop vite » avec la **checklist de relecture** (logique, performance, sécurité, licence), prouver chaque défaut par un test, le corriger, puis livrer TaskFlow avec un fichier d'instructions d'équipe et un dernier commit propre.

Point de départ

Les 8 fichiers de `suggestions/` sont des fonctions TaskFlow générées par Copilot. `revue.test.js` contient un exemple (suggestion 02) qui échoue.

Étapes

1. **Relire sans outil** (10 min) : pour chaque suggestion, notez dans `REVUE.md` le défaut soupçonné et sa catégorie : logique, performance, sécurité, API inventée, licence.
2. **Second avis** : sur deux suggestions de votre choix, sélectionnez le code, clic droit › Copilot › revue de code (ou chat Ask : « Fais la revue de ce code ») et comparez avec votre liste : qu'a-t-il trouvé ? Qu'a-t-il manqué ?
3. **Prouver** : pour 01 à 07, ajoutez un `describe` dans `revue.test.js` qui **échoue** sur la suggestion d'origine (`npm run test:revue`).
4. **Corriger** : corrigez chaque suggestion (vous-même ou avec `Ctrl+I`) jusqu'au vert. Pour 08, décidez dans `REVUE.md` : remplacer par `structuredClone` , et retirer le code copié.
5. **Instructions d'équipe** : créez `.github/copilot-instructions.md` (5 à 8 règles tirées de la formation : `node:test` , pas de `new Date()` dans la logique, `textContent` jamais `innerHTML` , aucun secret, JSDoc obligatoire...). Vérifiez dans le chat qu'elles sont prises en compte (référence affichée sous la réponse).
6. `npm test` tout au vert, commit « Atelier 8 : revue et instructions d'équipe ». Préparez 2 minutes de restitution (partage d'écran).

C'EST TERMINÉ QUAND...

- ☐ `REVUE.md` : 8 défauts identifiés, classés, avec la correction choisie.
- ☐ `npm run test:revue` : au moins 7 `describe` , tous au vert après correction.
- ☐ `.github/copilot-instructions.md` versionné ; `npm test` au vert.

PIÈGES À OBSERVER

- `sortBy` n'existe pas sur les tableaux JavaScript : une **API inventée** se repère vite... à condition d'exécuter le code.
- La suggestion 05 cumule trois problèmes : jeton en clair, jeton dans l'URL (journaux serveur), HTTP sans TLS.
- Copilot en mode revue est un **second avis** : il ne remplace ni vos tests ni la relecture humaine.

BONUS

Ajoutez une règle à `.github/copilot-instructions.md` , puis redemandez la suggestion 04 dans une nouvelle conversation : le code proposé respecte-t-il maintenant `textContent` ?

COINCÉ ?

Vous ne voyez pas le défaut d'une suggestion : exécutez-la avec un cas limite (liste vide, saisie « (», date de demain). Si rien ne casse, passez à la suivante et revenez-y.

Annexe · Aide-mémoire

Les 8 raccourcis (Windows / Linux)

`Tab`	accepter la suggestion
`Échap`	refuser
`Ctrl+→`	accepter mot par mot
`Alt+]` `Alt+[`	suggestion suivante / précédente
`Alt+\`	déclencher une suggestion
`Ctrl+I`	chat en ligne (modifier la sélection)
`Ctrl+Alt+I`	vue Chat
`Ctrl+N` (chat)	nouvelle conversation

macOS : Ctrl devient Cmd, Alt devient Option.

Le commentaire en 5 ingrédients

Entrée · Sortie · Règle · Exemple (dont un cas refusé) · Contrainte

Lire avant d'accepter : 5 questions

1. Fait-elle ce que j'ai demandé ?
2. Ce qu'elle appelle existe-t-il ?
3. Et les cas limites ?
4. A-t-elle des effets de bord ?
5. Puis-je l'expliquer ligne à ligne ?

Checklist de relecture

Logique

- [ ] Fait ce qui est demandé, tout ce qui est demandé
- [ ] Cas limites : vide, null, bornes, doublons
- [ ] Erreurs gérées, messages clairs
- [ ] Fonctions et API qui existent vraiment

Performances

- [ ] Pas de boucle dans une boucle évitable
- [ ] Pas de recalcul inutile dans une boucle
- [ ] Set ou Map pour les recherches répétées

Sécurité

- [ ] Aucun secret, aucun jeton dans le code
- [ ] Entrées utilisateur jamais injectées telles quelles
- [ ] Dépendance proposée : existe-t-elle, est-elle sûre ?
- [ ] Licence : code recopié d'un dépôt public ?